

Guión de presentación — TP3 Perceptrones

35 slides · qué decir en cada una

Cada entrada corresponde al número de slide del PDF `slides_presentacion.pdf`. Las cajas verdes incluyen información de respaldo extraída de las notas en `docs/notas/` para responder preguntas. La presentación arranca con Ej 1 (no hay sección de “Decisiones de diseño” al inicio: las decisiones se discuten dentro del bloque de Ej 2 una vez que ya está planteado el problema).

Índice

Tiempo estimado total: 18–22 minutos.

Bloque 1. Apertura

Slide 1.1. Slide 1 — Portada

Qué decir

- Buenas, somos *[nombres del grupo]*.
- Vamos a presentar nuestros resultados experimentales sobre tres ejercicios: detección de fraude por knowledge distillation, clasificación de dígitos manuscritos, y mejora de esa clasificación.
- La presentación está organizada en tres bloques, uno por ejercicio.

Bloque 2. Ejercicio 1 — Knowledge Distillation

Slide 2.1. Slide 2 — Divider Ej 1

Qué decir

- Empezamos con el Ej 1: **detección de fraude por knowledge distillation**.
- Es un perceptrón simple, pensado como prueba de concepto.

Slide 2.2. Slide 3 — Ej 1: el problema

Qué decir

- *Knowledge distillation* = replicar la salida de un BigModel pre-entrenado con un TinyModel (acá, un perceptrón simple).
- El target de entrenamiento es `big_model_fraud_probability`, una **probabilidad continua entre 0 y 1**.
- **Regla clave del enunciado:** `flagged_fraud` (la etiqueta binaria real) **nunca se usa para entrenar**; sólo aparece después como criterio de evaluación.

- Como la salida tiene que estar en $[0, 1]$, la activación natural es la **sigmoide**.

Slide 2.3. Slide 4 — Ej 1: análisis del dataset (tres reglas determinísticas)

Qué decir

- **Antes de entrenar nada** hicimos exploración del dataset.
- Encontramos tres features con **umbrales duros** que separan perfectamente las clases (todo lo de la derecha del umbral es fraude): `amount_usd > 500`, `quantity ≥ 10`, `items_viewed ≥ 15`.
- Cada regla individual marca cientos de filas con **cero falsos positivos**.
- Aplicadas como OR, dan **TP=695, FP=0, FN=174** sobre 7.500 filas: *precision 100 %*, *recall 80 %*, *accuracy 97.7 %*.

Info de respaldo (de la nota)

Por qué importa el análisis previo del dataset: Pearson y Spearman cercanos a 0 en `timestamp`, `device_screen_resolution` y `time_since_last_login_s` indican que esas tres features no aportan señal lineal ni monótona. Las descartamos para el perceptrón simple del Ej 1 porque, además, dos de ellas tienen magnitudes de orden 10^9 y 10^6 que dominarían el cálculo del gradiente sin normalización perfecta. (`decisiones_y_backing_teorico.md`)

Slide 2.4. Slide 5 — Ej 1: conclusión del análisis

Qué decir

- **El 80 % del fraude se etiqueta con tres if, sin un solo FP.**
- El 20 % restante (174 casos) son fraudes *sutiles*: respetan los umbrales y solo se detectan combinando features continuas.
- Tres implicancias para el TinyModel: (a) tiene que igualar a las tres reglas como línea de base (*recall* $\geq 80 \%$), (b) el valor agregado se mide en cuántos de los 174 fraudes recupera, (c) las discontinuidades duras (saltos de $\sim 6 \%$ a 100% al cruzar un entero) son geométricamente imposibles para una recta — ahí debería brillar la sigmoide.

Slide 2.5. Slide 6 — Ej 1: lineal vs no-lineal (MSE)

Qué decir

- Mismo dataset, K-fold = 5 estratificado, mismas features. La única diferencia entre los dos modelos es la **presencia de la sigmoide**.
- El no-lineal saca **59 % menos MSE**: $0,0110 \pm 0,0004$ vs $0,0266 \pm 0,0008$.
- **¿Por qué gana tan claro?** El target es una probabilidad acotada en $[0, 1]$. La sigmoide *naturalmente* mapea su salida a ese rango. El lineal puede producir valores fuera de $[0, 1]$ y el MSE penaliza fuerte esa salida fuera de dominio.

Slide 2.6. Slide 7 — Ej 1: pruebas sobre threshold

Qué decir

- Con el threshold default 0,5 las métricas se ven *horribles* (precision 0,33, F1 0,50). **No es el modelo, es el threshold.**
- Las salidas de la sigmoide se concentran arriba: la mediana de los scores de fraude es 0,99. Los no-fraudes *no* bajan a 0,05, bajan a 0,4–0,6. Por eso el 0,5 sobre-detecta.
- Hicimos un sweep y el **óptimo por F1** es 0,89: precision 0,886, recall 0,861, F1 0,873, accuracy 0,971.
- El modelo aprendió un buen **ranking**, no una calibración perfecta.

Slide 2.7. Slide 8 — Ej 1: recomendación de umbral

Qué decir

- La elección de threshold depende del **costo asimétrico** entre falso positivo y falso negativo.
- Comparamos cuatro thresholds en la tabla: el default 0,5 deja 1743 FP; el óptimo F1 (0,89) deja 96 FP; el 0,99 deja **cero** FP a costa de bajar el recall a 0,527.
- Conclusión: **el modelo entrega scores; la política de threshold es decisión de negocio.**

Bloque 3. Ejercicio 2 — Clasificación de dígitos

Slide 3.1. Slide 9 — Divider Ej 2

Qué decir

- Pasamos al Ej 2: **clasificación multiclase** de dígitos manuscritos con un MLP.
- Acá entran las decisiones de diseño que vamos a discutir en bloque.

Slide 3.2. Slide 10 — Ej 2: el problema

Qué decir

- Diez clases (dígitos 0-9). Train: 12.449 muestras. Test: 2.498.
- Cada muestra: vector de 784 floats $\in [0, 1]$ — son imágenes 28×28 *flatten*.
- **Arquitectura final:** [784, 100, 50, 10] con ReLU en las capas ocultas y softmax en la salida.
- **Regla del TP:** el test set se evalúa *una sola vez al final*. Toda la búsqueda de hiperparámetros es sobre `digits.csv`.

Slide 3.3. Slide 11 — Ej 2: fases de experimentación

Qué decir

- Workflow disciplinado en **dos fases**.
- **Fase 1** — *exploratoria*, $k_folds = 1$: sweeps secuenciales (arquitectura $\rightarrow$ optimizador $\rightarrow$ LR $\rightarrow$ batch size) para reducir el espacio de búsqueda barato.

- Se consolida el ganador en un `base.json`.
- **Fase 2** — *validación*, $k_folds = 5$: comparativas one-at-a-time para confirmar que las elecciones son robustas, no suerte de un solo split.
- Sólo después corre el `final_eval` sobre `digits_test.csv`, *una sola vez*.

Slide 3.4. Slide 12 — Ej 2: sweep de arquitectura

Qué decir

- Comparamos cinco arquitecturas: la base, una con misma forma pero un poco más ancha ([784, 128, 64, 10]), una *wider*, una *deeper* y una *shallow*.
- **Empate técnico** entre la base y [784, 128, 64, 10] ($\Delta < 0,4\%$, $\text{std} \sim 0,4\%$).
- **Wider** ([784, 200, 100, 10]) mejora val (+0.17 pp) pero *empeora test* (-0.48 pp) y es 3,6× más lento.
- **Decisión por parsimonia (regla de Occam)**: elegimos la más simple y rápida. *Wider* es el ejemplo empírico de la curva U: aumentar capacidad sin más datos $\Rightarrow$ overfitting.

Info de respaldo (de la nota)

Capacidad del modelo (slide 8 del PDF de regularización): “habilidad de un modelo de aproximar una variedad de funciones”. Cambia con: elección de modelo, arquitectura, cantidad de features de input. La curva U del slide 9 muestra que hay un punto óptimo: por debajo, underfitting; por encima, overfitting. *Wider* sin más datos cae en la zona de overfitting — es lo que vimos. (`decisiones_y_backing_teorico.md` §6)

Slide 3.5. Slide 13 — Ej 2: sweep de optimizador

Qué decir

- Tres optimizadores: SGD vanilla, Momentum ($\beta = 0,9$), Adam.
- **Adam gana**: 96,74 %, `best_epoch = 7`.
- Momentum queda dentro del desvío. SGD vanilla *no convergió en 50 epochs*.
- Elegimos Adam por convergencia más rápida y resultado igual o mejor.

Slide 3.6. Slide 14 — Función de activación

Qué decir

- Por contexto: sigmoide en Ej 1 (target [0, 1]); **ReLU** en capas ocultas de Ej 2/3; **softmax** en la salida (clasificación multiclase).
- **¿Qué es saturar?** Una activación satura cuando $f'(z) \rightarrow 0$ para entradas grandes. Sigmoide y tanh saturan en sus extremos: el gradiente que llega a las capas previas se desvanece (*vanishing gradient*) y la red deja de aprender.
- **ReLU** = $\max(0, z)$: derivada 1 en zona positiva, 0 en negativa. Sin saturación en su lado activo $\Rightarrow$ gradiente constante a través de muchas capas.

Info de respaldo (de la nota)

ReLU en detalle:

- Con buena inicialización, $\sim 50\%$ de las neuronas están apagadas en cada batch. Es *feature*, no *bug* (sparsity).
- *Neuronas muertas*: si una neurona devuelve 0 para todos los datos, su gradiente queda en 0 y nunca se actualiza. Suele pasar con learning rate alto o mala init.
- **He init** ($\sigma = \sqrt{2/\text{fan_in}}$) compensa que ReLU “mata” la mitad del input — mantiene varianza estable a través de capas. Por eso usamos He cuando hay ReLU y Xavier cuando hay tanh/sigmoid.

(relu.md)

Slide 3.7. Slide 15 — Inicialización de pesos

Qué decir

- **¿Por qué no cero?** Si todas las neuronas arrancan iguales, hacen lo mismo en cada capa. El gradiente las mueve igual — *nunca se diferencian*. Es el problema de simetría.
- **¿Por qué pequeños?** Pesos grandes saturan las activaciones.
- Tres esquemas: **He** ($\sigma = \sqrt{2/\text{fan_in}}$) para ReLU; **Xavier** ($\sigma = \sqrt{1/\text{fan_in}}$) para tanh/sigmoid/softmax; **Uniforme** $[-0,1, 0,1]$ para el perceptrón simple del Ej 1.

Info de respaldo (de la nota)

De dónde vienen las fórmulas: la cátedra mandó un HTML sobre weight initialization que cubre Xavier (Glorot & Bengio 2010) y He (He et al. 2015). Nuestra fórmula de Xavier usa $\sqrt{1/\text{fan_in}}$ (variante LeCun); el HTML cita la forma Glorot $\sqrt{2/(\text{n_in} + \text{n_out})}$. *Si preguntan:* ambas formas existen en la literatura (PyTorch tiene las dos), empíricamente quedaron en empate técnico ($\sim 96\%$, std $\sim 0,3\%$). (decisiones_y_backing_teorico.md §2)

Slide 3.8. Slide 16 — Optimizador

Qué decir

- **SGD vanilla:** $w \leftarrow w - \eta \nabla L$. Lo más simple. Lento en valles alargados.
- **Momentum** ($\beta = 0,9$): acumula *velocidad* en la dirección del gradiente; atraviesa valles sin oscilar.
- **Adam:** combina momentum con *learning rate adaptativo por parámetro*.
- Resultado en Ej 2 lo vimos en el sweep: Adam $\approx$ Momentum $\gg$ SGD. Elegimos Adam por convergencia más rápida.

Info de respaldo (de la nota)

Adam = Momentum + RMSProp: mantiene un promedio móvil del gradiente (como momentum) y otro del gradiente al cuadrado (como RMSProp). Divide el update por la raíz del segundo momento $\Rightarrow$ learning rate adaptativo por parámetro. Defaults: $\alpha = 10^{-3}$, $\beta_1 = 0,9$, $\beta_2 = 0,999$, $\varepsilon = 10^{-8}$. (decisiones_y_backing_teorico.md §3)

Slide 3.9. Slide 17 — Learning rate

Qué decir

- Recomendación de la clase: learning rate *pequeño* y probar varios órdenes de magnitud.
- Demasiado grande $\rightarrow$ la red oscila o diverge. Demasiado chico $\rightarrow$ no converge en tiempo razonable.
- Hicimos sweep en **dos pasos**: Fase 1 con cinco valores razonables; Fase 2 con extremos catastróficos para mapear los bordes.
- Las próximas dos slides muestran las tablas de cada fase.

Slide 3.10. Slide 18 — Learning rate: Fase 1 exploratoria (k=1)

Qué decir

- $lr = 10^{-3}$ ganó claro: 96,74 %, best_epoch = 7.
- $lr = 10^{-4}$ converge pero *muy lento* (best_ep = 47).
- $lr \geq 5 \cdot 10^{-3}$ corta a las 2-4 épocas: **overshoot**.
- Tres criterios apuntan al mismo número: máximo de val_acc, convergencia rápida y ausencia de overshoot.

Info de respaldo (de la nota)

Tabla de Fase 1.3 completa (5 valores, k=1):

- 0,001 $\rightarrow$ 96,74 %, best_ep=7. ✓ Óptimo.
- 0,0005 $\rightarrow$ 96,54 %, best_ep=17. Lento.
- 0,0001 $\rightarrow$ 96,22 %, best_ep=47. No converge.
- 0,005 $\rightarrow$ 96,02 %, best_ep=4. Overshoot moderado.
- 0,01 $\rightarrow$ 95,30 %, best_ep=2. Overshoot fuerte.

(lr_sweep_conclusion.md)

Slide 3.11. Slide 19 — Learning rate: Fase 2 validación (k=5)

Qué decir

- Mismo ganador, ahora con K-fold = 5: 10^{-3} **confirmado** (96,22 %).
- Los **extremos catastróficos** confirman la teoría: $lr = 0,1$ diverge (29 %), $lr = 10$ explota a NaN — la red queda en accuracy random (~ 12 %, equivalente a 1/10 clases).
- Robusto: dos sweeps independientes coinciden.

Slide 3.12. Slide 20 — Convergencia y bias

Qué decir

- Dos criterios distintos por la naturaleza de cada problema. **Ej 1:** ε absoluto sobre MSE de train (regla del apunte para perceptrón simple). **Ej 2/3:** *early stopping* sobre `val_loss` con `patience = 10`.
- Early stopping detecta overfitting que ε solo no detecta.
- **Bias trick:** agregamos columna de 1's al input de cada capa, los pesos quedan con shape `(n_out, n_in+1)`. El bias es un peso más, se actualiza con la misma regla.

Info de respaldo (de la nota)

Early stopping (slide 14 del PDF de regularización):

- En cada época trackeamos `best_val_loss` y guardamos los pesos del mejor modelo.
- Si la `val_loss` no mejora durante `patience` épocas seguidas, cortamos y *restauramos los pesos del mejor modelo* (no del último).
- `epochs` máximo en Ej 2/3 = 50 con `patience = 10`. `best_epoch` promedio ~ 5 — la red converge mucho antes de los 50.

(early_stopping.md)

Slide 3.13. Slide 21 — Ej 2: curvas de aprendizaje (K=5)

Qué decir

- **Convergencia limpia** en ~ 5 epochs.
- Train y validación bajan *juntos*: **sin overfitting visible**.
- Las bandas son los desvíos del K-fold; estrechas $\Rightarrow$ entrenamiento estable entre folds.

Slide 3.14. Slide 22 — Ej 2: matriz de confusión del base (val K=5)

Qué decir

- Diagonal limpia para **9 de 10** clases.
- **La clase 8 colapsa:** `precision = recall = 0`.
- Hint fuerte: algo raro pasa con la **distribución de los datos** (cobertura insuficiente del 8 en `digits.csv`).

Slide 3.15. Slide 23 — Ej 2: final eval, el cachetazo

Qué decir

- Aquí viene la sorpresa.
- $\text{val K-fold} = 96,22\%$. **Test** (`digits_test.csv`) = $86,30\%$. **Drop de 10 puntos porcentuales**.
- **No es underfitting, no es overfitting clásico**: train y val bajaron juntos. Es **distribution shift** — los datos de test son una población distinta de los de train.

Info de respaldo (de la nota)

¿**Qué es distribution shift**? La distribución de los datos de entrenamiento es distinta de la de evaluación: el modelo aprende patrones de train que no generalizan al test, no por memorizar (overfitting) sino porque *train y test son poblaciones distintas*.

- *Covariate shift*: $P_{\text{train}}(x) \neq P_{\text{test}}(x)$, pero $P(y|x)$ es la misma. Es lo que vimos acá.
- Síntomas en el TP3: drop val→test asimétrico, clase 8 colapsada, errores distribuidos (no concentrados).

(`explicacion_distribution_shift.md`)

Slide 3.16. Slide 24 — Ej 2: matriz sobre `digits_test.csv`

Qué decir

- Las confusiones están **distribuidas**, no concentradas en una clase. Ese patrón es síntoma de *shift*, no de falta de capacidad (capacidad insuficiente daría errores concentrados).
- Conclusión: **más datos, no más modelo**. Eso es lo que el Ej 3 provee con `more_digits.csv`.

Bloque 4. Ejercicio 3 — Mejorar la clasificación

Slide 4.1. Slide 25 — Divider Ej 3

Qué decir

- Cerramos con el Ej 3: **llevar el accuracy de test al 98 %**.
- Vamos a probar si la hipótesis del shift es correcta.

Slide 4.2. Slide 26 — Ej 3: hipótesis y plan

Qué decir

- **Hipótesis**: el techo del Ej 2 es por distribution shift, no por falta de capacidad.
- Plan secuencial: (1) **más datos** con `more_digits.csv` (15.742 muestras adicionales) — *cero cambios al modelo*; (2) si no llega al 98 %, agregamos regularización (L2, dropout, augmentation).

Slide 4.3. Slide 27 — Ej 3: el movimiento dominó

Qué decir

- Con *sólo* concatenar `more_digits.csv`: val 96,22 $\rightarrow$ 97,06, test 86,30 $\rightarrow$ 96,36.
- **+10 puntos porcentuales en test**, *cero cambios al modelo*.
- Macro F1 0,857 $\rightarrow$ 0,959. La clase 8 que estaba colapsada en Ej 2 ahora se predice correctamente.
- **Confirma la hipótesis del shift**: era cobertura, no capacidad.

Slide 4.4. Slide 28 — Ej 3: matriz de confusión del ganador (test)

Qué decir

- Diagonal mucho más limpia.
- Los errores residuales se reparten — síntoma de que **ya no hay un agujero específico de cobertura**, sino limitaciones globales del modelo.

Slide 4.5. Slide 29 — Ej 3: resultados

Qué decir

- Probamos siete configuraciones tomando `base_extra_data` como referencia (test 96,36).
- **L2** ($\lambda = 10^{-4}$): +0,20 pp consistente.
- **Dropout** ($p = 0,2$): -0,08 pp en test (gana en val, pierde en test).
- **Ganador: L2 + augmentation gaussiana** ($\sigma = 0,05$) $\rightarrow$ **96.88 %** en test (+0,52 pp sobre el baseline con extra data).
- *Wider + L2 + aug* no supera al ganador: descarta falta de capacidad.

Info de respaldo (de la nota)

Resumen de cada técnica (qué hace + cómo se implementa):

- **L2**: agrega $\frac{1}{2}\lambda\|w\|^2$ al loss; gradiente $+\lambda w$. *El bias no se penaliza* (la columna 0 de cada matriz).
- **Augmentation gaussiana**: en cada batch suma ruido $\mathcal{N}(0, \sigma^2)$ a las entradas. $\sigma = 0,05$ es lo que ganó.
- **Dropout**: cada neurona se “apaga” con probabilidad p durante training; en inferencia se usa la red entera. Inverted dropout: la división por $(1 - p)$ compensa la baja densidad de activaciones.

(resultados_ejercicio3_explicacion.md, clase_regularizacion_cosas_implementadas.md)

Slide 4.6. Slide 30 — Ej 3: comparación visual con Ej 2

Qué decir

- Línea roja: target $\geq 98\%$.
- **El salto principal viene del baseline con extra data**, no de la regularización fina.
- La regularización aporta un techo modesto ($\sim 0,5$ pp) sobre ese baseline.

Slide 4.7. Slide 31 — Ej 3: curvas del modelo ganador

Qué decir

- Convergencia más lenta que en Ej 2 base (best_epoch ~ 10 vs 5) por L2 + augmentation, que “cuestan” alguna época de optimización a cambio de mejor generalización.
- Sin overfitting visible.

Slide 4.8. Slide 32 — Ej 3: qué ayudó y qué no

Qué decir

- **Ayudó:** más datos (+10 pp, dominante), L2 (+0,20 pp), aug gaussiana (+0,32 pp adicional sobre L2 — *sinergia*).
- **No ayudó:** dropout (gana en val, pierde en test — reduce varianza al fold, no al shift), wider (mejor val, peor test — confirma que no falta capacidad), combinar todo (L2+dropout+aug peor que L2+aug solo).

Info de respaldo (de la nota)

Por qué dropout no transfirió: reduce la varianza al fold de cross-validation (mejora val K-fold) pero *no corrige distribution shift*. Si train y test son poblaciones distintas, dropout no pone ningún caso de test en la mesa de entrenamiento. Lección: una técnica que mejora val no necesariamente mejora test cuando hay shift. ([explicacion_distribution_shift.md](#))

Slide 4.9. Slide 33 — Ej 3: por qué no llegamos al 98 %

Qué decir

- Mejor test: 96,88 %. Faltan 1,12 pp.
- **Hipótesis principal:** el augmentation **gaussiano** simula ruido isotrópico por píxel. Pero el shift parece ser **geométrico** (rotación, traslación, grosor de trazo distintos por estilo de escritura).
- En la clase de regularización vimos **cuatro** formas de augmentation (gaussiano, rotaciones, traslaciones, escalas); sólo implementamos la primera.
- *Wider* empeoró $\Rightarrow$ no es problema de capacidad. Es problema de cobertura.
- **Próximo paso natural:** augmentation geométrico.

Info de respaldo (de la nota)

Matching con el PDF de regularización:

- Slide 14 — Early stopping: *implementado* y activo en Ej 2/3.
- Slide 18 — Augmentation: gaussiano (*implementado*, $\sigma = 0,05$), rotaciones / traslaciones / escalas (*NO implementados*).
- Slides 20-25 — L2 / Weight Decay: *implementado*, $\lambda = 10^{-4}$.
- Slide 26 — Dropout: *implementado*, *descartado por no transferir a test*.

Las tres formas de augmentation que NO implementamos son la hipótesis principal del techo en 96.88 %. (`clase_regularizacion_cosas_implementadas.md`)

Bloque 5. Cierre

Slide 5.1. Slide 34 — Conclusiones del TP

Qué decir

- Tres ideas para llevarse:
 - (1) **Una buena pipeline gana más que tunear hiperparámetros.** El +10 pp del Ej 3 vino de *datos*, no de modelo.
 - (2) **Los empates técnicos son más comunes que los wins claros.** Hay que elegir por simplicidad y criterio de complejidad en tiempo y espacio, no por décimas en val.
 - (3) **“Mejor en val” $\neq$ “mejor en test” cuando hay shift.** Dropout fue el ejemplo de manual.
- Status: Ej 1, Ej 2, Ej 3 completos. **Target $\geq 98\%$: no se alcanzó (96.88 %).** Lo documentamos con hipótesis y próximos pasos.

Slide 5.2. Slide 35 — Gracias

Qué decir

- Gracias. Quedamos atentos a las preguntas.